DAILY READING / 完整中文译本

当智能体替你工作时，如何保持批判性思考

Critical thinking when agents do the work

Addy Osmani

阅读与翻译 2026.11.09

本文目录

01 认知卸载、判断让渡与认知投降

02 我会问自己的问题

03 把验证变成习惯

04 真正关注生成出来的内容

05 高质量的输入，带来高质量的输出

06 “但模型会比我更擅长这些事”

07 日常可以采用的做法

08 值得带走的提醒

译本说明：全文依据用户提供的英文文本翻译，保留全部段落与章节顺序。正文中的“我”均指作者；阅读讨论单独保存。

来源：X / @addyosmani / 原始帖子

https://x.com/addyosmani/status/2117512471930167377

过去，人们教我们进行批判性思考，是要以谦逊的好奇心和适度的怀疑态度接触新信息：听取不同观点，形成自己的判断。这仍然成立。但现在，我的大量工作都通过智能体完成，日常工作中越来越没有什么会迫使我进行批判性思考了。对于那些已经全面拥抱智能体工作流的工程师和产品人员，也就是同时运行多个会话，并期待模型继续进步的人，我想先说明：我非常喜欢这些工具。写这篇文章，并不是为了提倡减少使用智能体工作流。

我的目标，是帮助你在一个任何事情都可以轻松交给智能体的世界里，保持批判性思考。

过去，我们大部分的认知加工与审美判断，都发生在提出问题与得到答案之间。如今，这段间隔已经缩短到以秒计算。智能体工作流用得越多，保持批判性思考就越困难，不过原因可能与人们预想的不同：随着模型进步，明显错误所占的比例在下降。很多输出，甚至可能是大多数输出，都会写得很好，而且基本正确，让人觉得没有必要再看一遍。

即使你同时运行许多智能体，查看这些输出的，也仍然只有你这一双眼睛。

你必须主动把批判性思考安排进日常工作，因为默认情况下，没有什么会替你这样做。

之所以很难自行纠正这种倾向，部分原因就在于智能体工作流让人感觉很愉快。在忙碌的一天里，迅速得到令人满意、而且经常正确的答案，让人精神一振。但也正因如此，我们很容易放松警惕。

认知卸载、判断让渡与认知投降

和别人讨论这个问题时，我会把它描述成一条连续的光谱：

最上端是认知卸载（cognitive offloading）：把任务委托给 AI，但仍然对结果负责。我每天都在这样做，你也应该这样做。中间是判断让渡（deferral）：工具的输出直接成了你的新输入，那个结果已经不再被当作有待检验的假设。最下端则是认知投降（cognitive surrender）：AI 的输出悄悄变成了你自己的看法，你不再觉得有必要检查它。

“认知投降”这个术语由沃顿商学院的 Steven Shaw 和 Gideon Nave 提出。它准确描述了我所看到的现象：随着信任增加，或者人变得更懒，人们会沿着这条光谱向下滑动。这种转变很少被察觉，因为每一步都很渐进，看起来也合情合理。你认真读了上一个回答，于是下一个就略读一遍。再下一个，你只扫一眼框架。随后，有人问起其中某一行，你却解释不出来，甚至请智能体帮忙也解释不清。

这就是认知投降，也是认知债务积累的方式。通常，直到你想运用对系统的心智模型，却发现自己根本没有这个模型时，才会意识到问题。Margaret-Anne Storey 曾写过，这种现象在团队中体现为：团队对系统运行原理的共同理解逐渐流失。我也写过它在代码层面的对应物，

即理解债务（comprehension debt），这个术语由 Jeremy Twei 提出。当下，这种债务完全不可见：代码库看起来整洁无瑕，测试也全部通过。直到你必须修改系统，却发现已经没有人理解它了，问题才会显现。同样的情况也会发生在你只看了战略文档或规格说明的目录，就签字批准的时候。

说清楚一点，我并不是要求大家做到不可能的事。要在脑中保留一个大型现代代码库的完整心智模型，本来就不可能。Claude Code 团队曾公开表示，他们会按需重建理解：需要理解哪一部分，就请 Claude 解释那一小块，而不是把完整模型一直记在脑中。正如我之前写过的，这类债务比大多数工程师想象的更容易偿还。只要能在需要时重建对局部的理解，你就不必始终掌握完整模型。你只需要清楚，自己缺少哪些理解。

对于风险较低的事情，尽管完全委托出去。但如果你无法解释一个问题的答案，甚至借助智能体也解释不清，那你就已经交出了对这个结果的掌控。

这也不总是问题。就在不久前，我录制一门课程，需要从三家评分相近的酒店里挑一家作为录制地点。我没有时间逐条查看所有评价，于是把这个决定交给了 AI。结果很好，下次我还会这样做。不过，在决定如何投资我们的钱、如何就孩子的职业选择给出建议时，我不会这样做；在工作中，面对我们应该构建什么、哪些改动会影响哪些用户、是否可以安全发布这些问题时，也不会。出错代价很低时，可以把事情交出去；如果后果要由别人承担，就要保留自己的推理与判断。

我会问自己的问题

当我不知道自己怎么看一件事时，通常会回到这样一组基本问题：我怎么知道自己在处理正确的问题？我怎么知道解决方式是对的？有没有证据支持我的结论？我怎么知道工作已经完成？大多数时候，无非就是谁、什么、哪里、为什么、何时和怎样。写下来显得幼稚得可笑，但它会迫使你从一些原本可能忽略的角度思考。如今，我们太容易直接跳过这一切，转而问智能体它觉得我们该怎么做。下面是我现在会问自己的这些问题的新版本。

提问之前，我有没有先形成一个假设？在让智能体处理任何不那么简单的任务之前，我都会先写下自己认为可能的答案，哪怕只是先在脑中想一下。猜得不好也没关系。有了新信息，你随时可以改变想法。不过，一旦读过一份写得很好的回答，就很难再想象其他可能。如果你完全没有方向，可以请模型做一次 Thariq Shihipar 所说的“盲点检查”（blind spot pass）：把你已经知道的告诉模型，让它找出那些你连自己不知道都没有意识到的东西。

我是在请它帮助我思考，还是请它替我承担思考？两种都可以，前提是你知道自己正在做哪一种。

在什么条件下，这个答案会是错的？然后就去查明这些条件是否成立。与一个你真正可以执行的测试相比，模型对自身置信度的评估价值小得多，所以给它一个实际测试。

我有没有从不同角度听取反方论证？当我对一个较为开放的问题持有强烈立场时，我会让不止一个模型站在相反一方进行论证，然后审阅输出，再形成自己的判断。这样花的时间更多，但我最终会对这个问题了解得多得多。

面对将来必须维护这项成果的人，我敢不敢为这个结果打包票？这适用于代码，也适用于你交付的任何东西。我认为，这就是能够对结果作出交代的标准。

我不再练习哪些能力了？也许是某些日常调试工作。我可以接受放下一部分，但希望这是有意识的选择。

把验证变成习惯

最近我写了很多关于验证的内容，因为我认为瓶颈已经从写代码，转移到了确认自己做得对不对。当你给智能体提供验证自身工作的手段，无论是端到端测试、性质测试，还是实际操作应用，它都可以完成一次修改，运行验证测试，读取结果，再次尝试，而这一切可能发生在你查看任何内容之前。这就是 Boris Cherny 所说的“给 Claude 一种验证自身工作的方法”，我认为这是取得出色结果最关键的要素之一。你的工作将是构建这些循环并审阅证据，而不必逐行阅读所有内容。

你也可能把验证这件事一并交出去。“有没有证据支持我的结论？”会变成“检查变绿了吗？”智能体让编写测试变得极其便宜，因此你会看到大量绿色结果；但一个通过的测试，可能根本抓不住已经损坏的功能。如果测试和功能犯了同样的错误，测试仍然会通过。而一个绿色检查只传达某一项性质的信息，我们却太容易想当然地以为它覆盖了一切。

当检查变绿时，先问它证明了什么，再问它没有证明什么。

提前定义什么叫“已经验证”，因为事后你会为任何结果找到合理化的解释。

智能体工作的终点应该是某种具体可见的东西：通过的测试、截图、日志或执行追踪。这能确保完成标准建立在事实之上，而不是根据“看起来已经做完了”之类的感觉来解释，甚至放弃判断。收到智能体的完成通知后，先不要看摘要，先看它实际执行的命令片段及其输出。

Claude Code 还可以通过一个具有全新上下文的子智能体，对自己的工作进行自我批评，作为合理性检查。值得把它固定放进工作循环，让它每次都运行，而不是想起来才做。我仍然会在最后问一句：“我有没有漏掉什么？”这听起来有点傻，但很多时候，我用前沿模型完成

复杂功能，查看过、审阅过之后，再问一句“我有没有漏掉什么？”，它就会再指出两三件事。所以我总会问。

当然，并不需要每次都如此严格；知道什么时候应该深入，本身就是一种能力。我们关于 Claude Code 团队如何做代码审查的文章中，我最喜欢的一条经验法则是：“如果你说不清这些检查会漏掉什么，就要求一次真正的审查，而不是一次快速批准。”

这也适用于代码之外的工作。如果智能体替你综合整理了五十次客户访谈，或者做了一份竞争分析，就抽查几份原始文档，确认总结准确。对于建议所依赖的某个数字，也要核实其计算是否正确。

智能体让建立验证机制变得便宜。对你而言，困难之处在于知道什么样的验证才算好。

真正关注生成出来的内容

假设智能体给你提交了一个 600 行的拉取请求（PR）。你大致看了看结构，没有逐行阅读。变量名看起来合理，测试也都通过，于是你批准了它。但在代码中间某处，事务边界内的操作顺序发生了细微变化，或者某个你从未想过的边界情况，其默认行为被反转了。也许可以说你确实审查过，但你只检查了自己原本就知道应该找什么的那些地方。

编写代码和审查代码是两种不同的技能；审查到了某个程度，就会变成例行盖章。所以，阅读差异（diff）时，要像审查非 AI 编写的代码一样认真；然后再看差异之外，看看智能体究竟替你作了哪些决定。在一个很长的会话里，它会替你作很多小决定：默认值是什么，哪些内容不在范围内，当两项需求冲突时又重新解释了哪一项。

在一份声称一切顺利的摘要里，你看不到这些，所以要让它列出自己作过的决定和假设。然后考考自己。随机挑几行问一问：这一行是怎么回事？那一行呢？对于文档，可以关掉它，试着凭记忆写出主要论点。对于长会话，有些人的一个习惯我很喜欢：让 Claude 写一份变更报告，并在末尾附上测验，规则是“只有测验全部答对，我才合并”。

对你交付的东西，要理解到足以为它作出解释和辩护的程度。因为总有一天，可能会有人问起。

智能体也可以帮助你守住注意力。例如，我有一个开源项目，会收到很多拉取请求。我会用智能体帮忙判断收到了哪些改动、哪些违背了我们的准则，以及哪些地方可能需要我关注。我很少让智能体自行决定这些事，但它们可以帮我缩小关注范围，然后我就能认真思考：这些改动是否在用正确的方式解决正确的问题。

还有产出量的问题，你必须对自己诚实。我把它称为编排税（orchestration tax）：智能体生成的内容，与那些你愿意署名认可的成果之间的差距。如果已完成的工作堆积得比你审阅得还快，那么在解决这个问题之前，再开一个会话也无济于事。你需要收紧检查机制，减少需要亲自介入的内容，或者放慢速度。

当你的产出多到已经无法为它们负责时，你就不再是在审查，而是在心存侥幸。

高质量的输入，带来高质量的输出

人的心智模型仍然参与其中，只是位置从中间转移到了两端。你把精力投入任务前端的描述与上下文，以及后端的结果审查。这就是为什么好的输入对输出质量如此重要，也是为什么我把编写高质量提示视为一种批判性思考能力。

过去，如果你提出一个含糊的请求，别人会追问，你可能因此意识到："等等，其实我自己还没想清楚。"智能体往往不会在这里拦住你。它会补齐你省略的内容，交回一份看起来已经完成的成果，尽管背后的想法仍然模糊。我曾把这种现象写成意图债务（intent debt）：那些存在于并非由你亲自编写的成果中的隐含目标、约束和理由。与其他类型的债务不同，这一种没有别人能替你解决。意图必须来自你自己。

你的提示反映了你对问题的理解。如果你无法清楚表达它，那它在你自己脑中也可能还不清楚。

在求助之前，先用自己的话写一份需求简述。说明受众、不在范围内的事项，以及完成标准。如果第二轮纠正后仍然没有达到目标，就先停止小修小补，重新写你的提示。

你也可以让智能体帮助完成这个过程。我很喜欢 Andy Clark 对"把事情委托给 AI 系统"和"与 AI 系统协作"所作的区分。后一种情形里，你们进入了相互增强的良性循环：你的提示让输出更准确，输出又让下一次提示更准确，进而让你对问题的心智模型更加清晰。如果还不清楚，就让智能体围绕它不理解的地方采访你。如果结束会话时，你自己的心智模型也得到了完善，那就是成功的信号。

还要留意输入内容。每个智能体都很容易只是把前一个智能体的摘要放进自己的上下文；很快，到了第三个智能体，它可能已经完全没有读过原始文档。

你还需要拥有自己的输出标准。我在许多不同的创作媒介中尝试过 AI，其中大部分成果从未公开展示，因为它们达不到我的标准。现在，所有东西都能以漂亮的格式和自信的口吻呈现出来，因此，精美的外表已经不再是有用的质量信号。

AI 或许让产出更快了，但如果我不愿意把某项输出署上自己的名字，我就不会把它交出去。

某项成果达不到你的标准时，把原因记录到共享配置、钩子（hook）或测试里，避免同样的问题再次发生。品味就是这样逐渐积累的；记录某项成果为何没有达到要求，本身也是很好的练习。

“但模型会比我更擅长这些事”

如果你像我身边的许多人一样看好这个领域的未来，可能在前面几节就已经想到这一点：再过一两年，模型就能发现我能发现的那些问题了，那为什么还要费心保留这项能力？

对于许多问题的发现，你可能是对的。我自己也一直主张，把越来越多的检测工作交给智能体。但我仍然认为这很重要，原因在于：最终，总要有人对发布出去的东西负责。无论 AI 帮了多少忙，都必须有人承担责任。

对于某个具体任务，应该在多大程度上信任智能体，本身就是一项判断。要让自己的判断保持准确，审查工作成果是其中的一部分，因为模型会犯哪些错误，一直在变化。Anthropic 对自家工程师和研究人员开展的研究，也指出了这个陷阱，并称之为监督悖论（paradox of supervision）：“监督 Claude 所需要的，恰恰是那些可能因过度使用 AI 而退化的编程技能。”

下面这些问题对我最重要，而更强的模型也无法替我解决：你在构建什么？为什么值得做？会影响谁？你愿意为此牺牲什么？这些都需要由你来回答。智能体可以帮你把它们想清楚，我也希望它能帮上忙，但归根结底，它们仍然是你的问题。

你可以把工作以及很大一部分审查交出去，但不能把责任也交出去。

日常可以采用的做法

你不需要严格执行所有这些做法。挑选适合自己工作流的部分，加入其中就好。

每周一次，不用智能体，自己写一点代码；或者亲手完成一件真实的任务，那种如果忘了怎么做，会让你很难受的任务。我把这称为校准练习，但它不是一种道德要求。每个月一次，写下自己已经不再做哪些事，再决定其中哪些是你可以接受不再做的。我在一篇旧博客中提出的问题，到今天仍然有效：如果 AI 不存在，你会有多么不知所措？

如果某件事超出了你的理解，就去学会它。写作时，如果遇到一个之前没见过的短语，我当然可以在 Google 上找一句解释，然后把它写进文章。但我更愿意稍作研究，真正理解它，这样才能知道它应该放在什么位置、与其他内容有什么关系。

认真检查机器的工作成果，本身就是一项真正的工作。假装它不需要投入，最终就会让团队根本不去做。它不是免费的，所以要把它单独列入工作安排，这样你才真的会去完成。

不再练习的技能会逐渐消失，而且很可能比你想象的更快。对于那些你不愿放弃的技能，要继续亲手练习。

值得带走的提醒

有意识地行动。智能体可以考你、采访你、向你展示证据，但前提是你安排了这些环节。所以，把它们安排进去。

诚实地面对自己在这条光谱上的位置。如果你无法解释答案，甚至借助智能体也做不到，那你就已经放弃了对它的判断。

先有一个假设。提问之前先形成假设，哪怕它很差。

一个绿色检查只说明一件事。它检查了什么？又没有检查什么？

寻找证据。先看它运行了什么、返回了什么，再看摘要。

阅读时要弄清它作了哪些决定。它替你决定了什么？

认真编写提示。你如何表达自己的请求，体现了你对问题的理解程度。

设定标准，并坚持它。如果你不愿意为某项成果署名，就不要把它交出去。

通过亲手运用那些重要的技能，让自己始终参与其中。你可以先试一件事：提问前，写下你认为会发生什么；智能体完成后，写下它没有检查的一件事。告诉我，这是否改变了你阅读输出的方式。独立思考，提出更好的问题，并对结果负责。